

Tích hợp Chế độ Auto (SCL) vào OB1 (Ladder)

Hướng dẫn chi tiết — Dự án KLTN PLC_1

1. Tổng quan kiến trúc

OB1 (Ladder) đóng vai trò khung chính (chiếc xe). FB_Auto_Main (SCL) là một khối chức năng con (hệ thống phanh) — được OB1 gọi như một thành phần, không can thiệp vào logic điều khiển khác.

Nguyên tắc cốt lõi:

- Manual FB2 và Auto FB_Auto_Main đều ghi ra biến trung gian trong Flags [DB4], KHÔNG ghi thẳng ra Q.
- Một Network riêng cuối OB1 mới ghi từ Flags ra Q0.0/Q0.1/Q0.2 — đảm bảo chỉ 1 nguồn ghi mỗi output.
- SCL tự quản lý TriggerReq, FIFO, state machine — OB1 chỉ truyền input/output.

2. Thay đổi trong Flags [DB4]

Thêm 6 biến mới vào cuối DB4 (sau DebugState):

Tên biến	Kiểu	Mục đích
Auto_Conveyor	Bool	Output trung gian Conveyor từ chế độ Auto
Auto_Cylinder1	Bool	Output trung gian Cylinder1 từ chế độ Auto
Auto_Cylinder2	Bool	Output trung gian Cylinder2 từ chế độ Auto
Manual_Conveyor	Bool	Output trung gian Conveyor từ chế độ Manual
Manual_Cylinder1	Bool	Output trung gian Cylinder1 từ chế độ Manual
Manual_Cylinder2	Bool	Output trung gian Cylinder2 từ chế độ Manual

3. Tạo mới FB_Auto_Main (SCL) — v0.3

Tạo Function Block mới trong TIA Portal, đặt tên FB_Auto_Main, ngôn ngữ SCL. Tạo instance DB mới: DB_AutoMain [DB6]. Dán toàn bộ code sau:

```
FUNCTION_BLOCK FB_Auto_Main
{ S7_Optimized_Access := 'TRUE' }
VERSION : 0.3
```

```
VAR_INPUT
    SS0      : BOOL;    // Cảm biến vị trí camera (I0.3)
    SS1      : BOOL;    // Cảm biến vị trí Cylinder1 (I0.4)
    SS2      : BOOL;    // Cảm biến vị trí Cylinder2 (I0.5)
    PC_DataReady : BOOL; // PC báo kết quả AI đã sẵn sàng
```

```

    PC_KetQua      : INT;      // Kết quả AI:
0=WAIT,1=OK,2=NG_L,3=NG_H,4=MISSING
    ResetFault    : BOOL;      // Reset lỗi từ HMI/OB1 (Flags.ResetFault)
END_VAR

VAR_OUTPUT
    Conveyor      : BOOL;      // Lệnh băng tải → Flags.Auto_Conveyor
    Cylinder1      : BOOL;      // Lệnh Cylinder1 → Flags.Auto_Cylinder1
    Cylinder2      : BOOL;      // Lệnh Cylinder2 → Flags.Auto_Cylinder2
    PLC_TriggerReq : BOOL;      // Yêu cầu PC chụp ảnh → DB_PUT.PLC_TriggerReq
    ErrorActive    : BOOL;      // Có lỗi nghiêm trọng → Flags.ErrorActive
    ErrorCode      : INT;      // Mã lỗi: 301=Overflow,302=Underflow,401=AI
invalid
    QueueCount     : INT;      // Số sản phẩm đang trong FIFO →
Flags.QueueCount
    DebugState     : INT;      // State hiện tại → Flags.DebugState
(0,10,20,30,40,90)
END_VAR

VAR
    // --- Bắt cạnh cảm biến ---
    prevSS0 : BOOL;      // Giá trị SS0 scan trước
    prevSS1 : BOOL;      // Giá trị SS1 scan trước
    prevSS2 : BOOL;      // Giá trị SS2 scan trước
    ss0Rise : BOOL;      // Cạnh lên SS0 – kích hoạt AI
    ss0Fall : BOOL;      // Cạnh xuống SS0 – clear TriggerReq
    ss1Rise : BOOL;      // Cạnh lên SS1 – quyết định Cylinder1
    ss2Rise : BOOL;      // Cạnh lên SS2 – quyết định Cylinder2

    // --- AI Handshake ---
    triggerCycleActive : BOOL; // Đang trong chu kỳ chờ AI (SS0 đã lên,
chưa xuống)
    aiConsumedThisCycle : BOOL; // Đã nhận kết quả AI 1 lần trong chu kỳ
này

    // --- FIFO ---
    fifo          : FB_FIFO10; // Instance FIFO 10 phần tử
    headValue     : INT;      // Kết quả AI của sản phẩm đầu hàng
    fifoOverflow  : BOOL;      // FIFO báo đầy
    fifoUnderflow : BOOL;      // FIFO báo pop khi rỗng
    pushReq       : BOOL;      // Yêu cầu push kết quả mới vào FIFO
    pushResult    : INT;      // Giá trị cần push
    popReq        : BOOL;      // Yêu cầu pop sau khi sản phẩm xử lý xong

    // --- Cylinder sequences ---
    cyl1SeqState : INT;      // State sequence Cylinder1
(0=idle,1=extend,2=retract)
    cyl2SeqState : INT;      // State sequence Cylinder2
(0=idle,1=extend,2=retract)
    cyl1Done     : BOOL;      // Cylinder1 hoàn thành 1 chu kỳ (DonePulse)
    cyl2Done     : BOOL;      // Cylinder2 hoàn thành 1 chu kỳ (DonePulse)
    cyl1Busy     : BOOL;      // Cylinder1 đang hoạt động
    cyl2Busy     : BOOL;      // Cylinder2 đang hoạt động
    actCyl1Req   : BOOL;      // Yêu cầu kích Cylinder1 (one-shot)
    actCyl2Req   : BOOL;      // Yêu cầu kích Cylinder2 (one-shot)
    CylTimer1    : TON;      // Timer TON cho Cylinder1 (T#2S)
    CylTimer2    : TON;      // Timer TON cho Cylinder2 (T#2S)

    // --- Lỗi ---
    errOverflow   : BOOL;      // FIFO overflow từ handshake
    errInvalidResult : BOOL;   // Kết quả AI ngoài 0..4

    // --- State machine ---
    state : INT;      //
0=IDLE,10=RUN,20=WAIT_SS2,30=ACT_CYL1,40=ACT_CYL2,90=FAULT
END_VAR

```

```

VAR CONSTANT
    ST_IDLE      : INT := 0;
    ST_RUN       : INT := 10;
    ST_WAIT_SS2  : INT := 20;
    ST_ACT_CYL1  : INT := 30;
    ST_ACT_CYL2  : INT := 40;
    ST_FAULT     : INT := 90;
END_VAR

BEGIN
    // ---- 1) Bắt cạnh ----
    ss0Rise := SS0 AND NOT prevSS0;
    ss0Fall := NOT SS0 AND prevSS0;
    ss1Rise := SS1 AND NOT prevSS1;
    ss2Rise := SS2 AND NOT prevSS2;
    prevSS0 := SS0; prevSS1 := SS1; prevSS2 := SS2;

    // ---- 2) AI Handshake ----
    pushReq := FALSE; errOverflow := FALSE; errInvalidResult := FALSE;

    IF ss0Rise THEN
        IF QueueCount >= 10 THEN
            errOverflow := TRUE;
        ELSE
            PLC_TriggerReq      := TRUE;
            triggerCycleActive  := TRUE;
            aiConsumedThisCycle := FALSE;
        END_IF;
    END_IF;

    IF PC_DataReady AND (NOT aiConsumedThisCycle) AND triggerCycleActive
    THEN
        IF (PC_KetQua < 0) OR (PC_KetQua > 4) THEN
            errInvalidResult := TRUE;
        ELSE
            pushReq           := TRUE;
            pushResult        := PC_KetQua;
            aiConsumedThisCycle := TRUE;
        END_IF;
    END_IF;

    IF triggerCycleActive AND ss0Fall THEN
        PLC_TriggerReq      := FALSE;
        triggerCycleActive  := FALSE;
        aiConsumedThisCycle := FALSE;
    END_IF;

    // ---- 3) State machine ----
    popReq := FALSE; actCyl1Req := FALSE; actCyl2Req := FALSE;

    CASE state OF
        ST_IDLE:
            Conveyor := FALSE;

        ST_RUN:
            Conveyor := TRUE;
            IF ss1Rise THEN
                IF headValue = 2 THEN
                    actCyl1Req := TRUE; state := ST_ACT_CYL1;
                ELSE
                    state := ST_WAIT_SS2;
                END_IF;
            END_IF;

        ST_WAIT_SS2:
    
```

```

        Conveyor := TRUE;
        IF ss2Rise THEN
            IF (headValue = 3) OR (headValue = 4) THEN
                actCyl2Req := TRUE; state := ST_ACT_CYL2;
            ELSE
                popReq := TRUE; state := ST_RUN;
            END_IF;
        END_IF;

    ST_ACT_CYL1:
        Conveyor := FALSE;
        IF cyl1Done THEN popReq := TRUE; state := ST_RUN; END_IF;

    ST_ACT_CYL2:
        Conveyor := FALSE;
        IF cyl2Done THEN popReq := TRUE; state := ST_RUN; END_IF;

    ST_FAULT:
        Conveyor := FALSE; PLC_TriggerReq := FALSE;
        IF ResetFault THEN state := ST_IDLE; END_IF;

    ELSE state := ST_IDLE;
END_CASE;

// ---- 4) Cylinder sequences ----
FC_Cylinder1_Seq(
    StartAction := actCyl1Req,
    SeqState := cyl1SeqState,
    CylTimer := CylTimer1,
    Cylinder1Cmd => Cylinder1,
    Busy => cyl1Busy,
    DonePulse => cyl1Done
);
FC_Cylinder2_Seq(
    StartAction := actCyl2Req,
    SeqState := cyl2SeqState,
    CylTimer := CylTimer2,
    Cylinder2Cmd => Cylinder2,
    Busy => cyl2Busy,
    DonePulse => cyl2Done
);

// ---- 5) FIFO ----
fifo(
    Push := pushReq,
    PushValue := pushResult,
    Pop := popReq,
    HeadValue => headValue,
    Count => QueueCount,
    Overflow => fifoOverflow,
    Underflow => fifoUnderflow
);

// ---- 6) Fault ----
ErrorActive := FALSE; ErrorCode := 0;
IF errOverflow OR fifoOverflow THEN
    ErrorActive := TRUE; ErrorCode := 301;
ELSIF fifoUnderflow THEN
    ErrorActive := TRUE; ErrorCode := 302;
ELSIF errInvalidResult THEN
    ErrorActive := TRUE; ErrorCode := 401;
END_IF;

IF ErrorActive THEN state := ST_FAULT; END_IF;
DebugState := state;
END_FUNCTION_BLOCK

```

4. Sửa OB1 Ladder — từng Network

Network 1 — Giữ nguyên

Không thay đổi. Logic Start-Stop và Lamp vẫn hoạt động bình thường.

Network 2 — Giữ nguyên

Không thay đổi. Logic chọn chế độ Auto/Manual.

Network 3 — Sửa: thêm điều kiện Manual cho TriggerReq

Không xóa dòng TriggerReq mà thêm điều kiện I0.2 (btManual) nối tiếp:

```
I0.3 (SS0) --| |-- I0.2 (btManual) --| |---- DB2.DBX0.3 (PLC_TriggerReq)
```

Lý do: chế độ Manual không có handshake AI nên cần ladder lo TriggerReq. Chế độ Auto thì I0.2=FALSE nên dòng này không active — SCL tự quản lý TriggerReq qua handshake, không xung đột.

Giữ nguyên 2 dòng còn lại:

- I0.4 (SS1) → DB2.DBX0.4 (PLC_Sensor1)
- I0.5 (SS2) → DB2.DBX0.5 (PLC_Sensor2)

Network 4 — Sửa: Manual output ra Flags thay vì Q trực tiếp

Điều kiện EN giữ nguyên: DB2.DBX0.1 (PLC_Manual) AND I0.2 (btManual)

Sửa mapping output của FB2 Mode_Manual:

Output FB2	Ghi vào
conveyor	"Flags".Manual_Conveyor (thay vì Q0.0)
cylinder_1	"Flags".Manual_Cylinder1 (thay vì Q0.1)
cylinder_2	"Flags".Manual_Cylinder2 (thay vì Q0.2)

Network 5 — MỚI: Gọi FB_Auto_Main

Điều kiện EN: DB2.DBX0.2 (PLC_Running) AND I0.1 (btAuto)

Tạo instance DB_AutoMain [DB6] cho FB_Auto_Main. Mapping input/output:

Pin FB_Auto_Main	Hướng	Kết nối
SS0	← Input	I0.3
SS1	← Input	I0.4
SS2	← Input	I0.5

PC_DataReady	← Input	"DB_GET".PC_DataReady
PC_KetQua	← Input	"DB_GET".PC_KetQua
ResetFault	← Input	"Flags".ResetFault
Conveyor	→ Output	"Flags".Auto_Conveyor
Cylinder1	→ Output	"Flags".Auto_Cylinder1
Cylinder2	→ Output	"Flags".Auto_Cylinder2
PLC_TriggerReq	→ Output	"DB_PUT".PLC_TriggerReq
ErrorActive	→ Output	"Flags".ErrorActive
ErrorCode	→ Output	"Flags".ErrorCode
QueueCount	→ Output	"Flags".QueueCount
DebugState	→ Output	"Flags".DebugState

Network 6 — MỚI: Output thật ra Q

Network này là nguồn DUY NHẤT ghi ra Q. Dùng tiếp điểm OR:

Nguồn Auto	Nguồn Manual	Output Q
"Flags".Auto_Conveyor	"Flags".Manual_Conveyor	Q0.0 (Conveyor)
"Flags".Auto_Cylinder1	"Flags".Manual_Cylinder1	Q0.1 (Cylinder1)
"Flags".Auto_Cylinder2	"Flags".Manual_Cylinder2	Q0.2 (Cylinder2)

Cách viết trong Ladder (ví dụ Conveyor):

```
"Flags".Auto_Conveyor --| |---+
                                +---- Q0.0 (Conveyor)
"Flags".Manual_Conveyor--| |---+
```

Làm tương tự cho Cylinder1 và Cylinder2.

5. Checklist thực hiện theo thứ tự

1. Mở Flags [DB4] → thêm 6 biến mới (Auto_Conveyor, Auto_Cylinder1, Auto_Cylinder2, Manual_Conveyor, Manual_Cylinder1, Manual_Cylinder2) kiểu Bool vào cuối.
2. Tạo FB_Auto_Main mới (SCL) → paste code Section 3 → compile.
3. Tạo instance DB_AutoMain [DB6] từ FB_Auto_Main.
4. OB1 Network 3: sửa dòng TriggerReq — thêm điều kiện I0.2 (btManual) nối tiếp, chỉ ghi DB2.DBX0.3 khi SS0=TRUE AND btManual=TRUE.
5. OB1 Network 4: sửa output Manual FB2 ghi vào Flags thay vì Q.
6. OB1 thêm Network 5: gọi FB_Auto_Main với điều kiện và mapping như bảng.
7. OB1 thêm Network 6: 3 coil Q0.0/Q0.1/Q0.2 với 2 tiếp điểm OR (Auto + Manual).
8. Compile toàn bộ → Download → Test chế độ Manual trước, sau đó Auto.

6. Mã trạng thái Auto để quan sát (DebugState)

Giá trị	Tên state	Ý nghĩa
0	ST_IDLE	Chờ — chưa nhấn Start hoặc đã dừng
10	ST_RUN	Đang chạy — băng tải ON, chờ sản phẩm
20	ST_WAIT_SS2	Sản phẩm qua SS1 (không NG_L), chờ SS2
30	ST_ACT_CYL1	Đang kích Cylinder1 (loại NG_L tại SS1)
40	ST_ACT_CYL2	Đang kích Cylinder2 (loại NG_H/MISSING tại SS2)
90	ST_FAULT	Lỗi — băng tải dừng, chờ ResetFault

7. Mã lỗi (ErrorCode)

ErrorCode	Nguyên nhân
301	FIFO overflow — hơn 10 sản phẩm chưa xử lý trên băng tải
302	FIFO underflow — pop khi FIFO rỗng (logic sai)
401	Kết quả AI ngoài khoảng 0..4 (PC gửi sai)

8. Hướng dẫn test với phần cứng

8.1 Chuẩn bị trước khi test

- Compile toàn bộ project TIA Portal — không còn lỗi đỏ.
- Download xuống PLC_1 (CPU 1212C DC/DC/DC).
- Mở Watch Table trong TIA Portal — thêm các biến: Flags.DebugState, Flags.ErrorActive, Flags.ErrorCode, Flags.QueueCount, DB_PUT.PLC_TriggerReq, DB2.DBX0.0 (PLC_Auto), DB2.DBX0.1 (PLC_Manual), DB2.DBX0.2 (PLC_Running).
- PC chạy chương trình Python AI ở chế độ chờ (lắng nghe DB_GET.PC_DataReady).

8.2 Test chế độ Manual trước (không cần AI)

Mục tiêu: xác nhận băng tải, Cylinder1, Cylinder2 hoạt động đúng trước khi thêm Auto.

#	Thao tác	Kết quả mong đợi	Kiểm tra tại
1	Nhấn btManual (I0.2=TRUE)	DB2.DBX0.1=TRUE, DB2.DBX0.0=FALSE	Watch Table
2	Nhấn Start-Stop (I0.0)	DB2.DBX0.2 (PLC_Running)=TRUE, Lamp_On=TRUE	Watch Table + đèn
3	Tác động SS0 (I0.3)	DB2.DBX0.3 (TriggerReq)=TRUE khi SS0=1, =FALSE khi SS0=0	Watch Table

4	Tác động SS1 (I0.4) khi PC_Cylinder1=TRUE từ Python	Q0.1=TRUE, Cylinder1 ra, sau 2s tự về	Phần cứng + Watch
5	Tác động SS2 (I0.5) khi PC_Cylinder2=TRUE từ Python	Q0.2=TRUE, Cylinder2 ra, sau 2s tự về	Phần cứng + Watch
6	Nhấn btManual lần nữa (I0.2=FALSE)	PLC_Running=FALSE, băng tải dừng	Watch Table

8.3 Test chế độ Auto — từng bước

Chỉ bắt đầu test Auto sau khi Manual đã pass hết.

Bước 1 — Khởi động Auto

#	Thao tác	Kết quả mong đợi	Kiểm tra tại
1	Nhấn btAuto (I0.1=TRUE), btManual=FALSE	PLC_Auto=TRUE, PLC_Manual=FALSE	Watch Table
2	Nhấn Start-Stop (I0.0)	PLC_Running=TRUE, DebugState=10 (ST_RUN), Q0.0=TRUE (băng tải chạy)	Watch + phần cứng
3	Nhấn Start-Stop lần 2	PLC_Running=FALSE, DebugState=0 (ST_IDLE), băng tải dừng	Watch + phần cứng

Bước 2 — Test AI Handshake tại SS0

#	Thao tác	Kết quả mong đợi	Kiểm tra tại
1	Khởi động Auto (PLC_Running=TRUE)	DebugState=10, băng tải chạy	Watch Table
2	Đưa sản phẩm qua SS0 (I0.3 lên rồi xuống)	TriggerReq=TRUE khi SS0=1, Python nhận tín hiệu chụp ảnh	Watch + log Python
3	Python xử lý xong, ghi PC_KetQua và PC_DataReady=TRUE	QueueCount tăng lên 1, TriggerReq=FALSE sau khi SS0 xuống	Watch Table
4	Lặp lại 3 lần với 3 sản phẩm	QueueCount=3, FIFO chứa đúng 3 kết quả theo thứ tự	Watch Table

Bước 3 — Test phân loại từng trường hợp

Kịch bản	PC_KetQua gửi	SS1 tác động	SS2 tác động	Kết quả mong đợi
OK	1 (OK)	Có	Có	Không kích cylinder nào, QueueCount giảm 1 tại SS2
NG_L	2 (NG_L)	Có	Không cần	Cylinder1 ra 2s rồi về, DebugState=30→10

NG_H	3 (NG_H)	Có	Có	Qua SS1 không kích, tới SS2 Cylinder2 ra 2s rồi về, DebugState=40→10
MIS SIN G	4 (MISSING)	Có	Có	Tương tự NG_H — Cylinder2 kích tại SS2

Bước 4 — Test xử lý lỗi

#	Tình huống	Kết quả mong đợi	Phục hồi
1	Python gửi PC_KetQua = 9 (ngoài 0..4)	ErrorActive=TRUE, ErrorCode=401, DebugState=90, băng tải dừng	Set Flags.ResetFault=TRUE → DebugState về 0
2	Đưa 11 sản phẩm qua SS0 không xử lý	ErrorActive=TRUE, ErrorCode=301 (overflow), băng tải dừng	Set Flags.ResetFault=TRUE
3	Dừng Python giữa chừng (không gửi kết quả)	TriggerReq giữ TRUE cho đến khi SS0 xuống, QueueCount không tăng	Khởi động lại Python

9. Đánh giá hệ thống

9.1 Tiêu chí đánh giá

#	Tiêu chí	Mức đạt	Cách đo
1	Chuyển chế độ Auto/Manual không gây xung đột output	Không bao giờ có 2 chế độ active cùng lúc	Quan sát Q0.0/Q0.1/Q0.2 khi chuyển chế độ
2	Phân loại đúng 4 loại sản phẩm	100% trường hợp test đúng cylinder	Chạy 20 sản phẩm, đếm sai sót
3	FIFO không bị lệch thứ tự	QueueCount đúng, cylinder kích đúng sản phẩm	Đánh số sản phẩm, theo dõi từng cái
4	Phát hiện và dừng khi có lỗi	ErrorActive=TRUE trong <1 scan sau khi lỗi xảy ra	Gửi PC_KetQua=9, quan sát Watch Table
5	Reset lỗi hoạt động	Sau ResetFault, hệ thống về ST_IDLE và chạy lại bình thường	Test sau mỗi tình huống lỗi
6	Timer cylinder đúng 2 giây	Cylinder ra đúng 2s (dùng đồng hồ hoặc Watch ET của CylTimer)	Đo ET trong Watch Table

9.2 Bảng ghi kết quả test

In bảng này ra và điền tay trong quá trình test:

L à n	Loại SP (KetQua)	Cylinder kích	DebugState cuối	QueueCount	Pass/Fail
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					

9.3 Các vấn đề thường gặp khi test

Hiện tượng	Nguyên nhân thường gặp	Hướng xử lý
DebugState không chuyển từ 0 sang 10	PLC_Running=FALSE — chưa nhận Start hoặc btAuto chưa active	Kiểm tra I0.1, DB2.DBX0.2 trong Watch Table
QueueCount không tăng sau khi SS0 tác động	Python chưa gửi PC_DataReady=TRUE hoặc PC_KetQua sai	Kiểm tra log Python, kiểm tra DB_GET trong Watch
Cylinder kích sai loại sản phẩm	FIFO bị lệch — pop/push không đồng bộ	Reset hệ thống, chạy lại từ 1 sản phẩm, theo dõi QueueCount từng bước
Băng tải dừng đột ngột, ErrorCode=301	Python xử lý chậm, nhiều sản phẩm dồn vào SS0 trước khi có kết quả	Giảm tốc độ băng tải hoặc tăng tốc xử lý AI
Cylinder ra nhưng không về	CylTimer không chạy — SeqState bị kẹt ở 1	Kiểm tra CylTimer1.ET trong Watch Table, xem SeqState